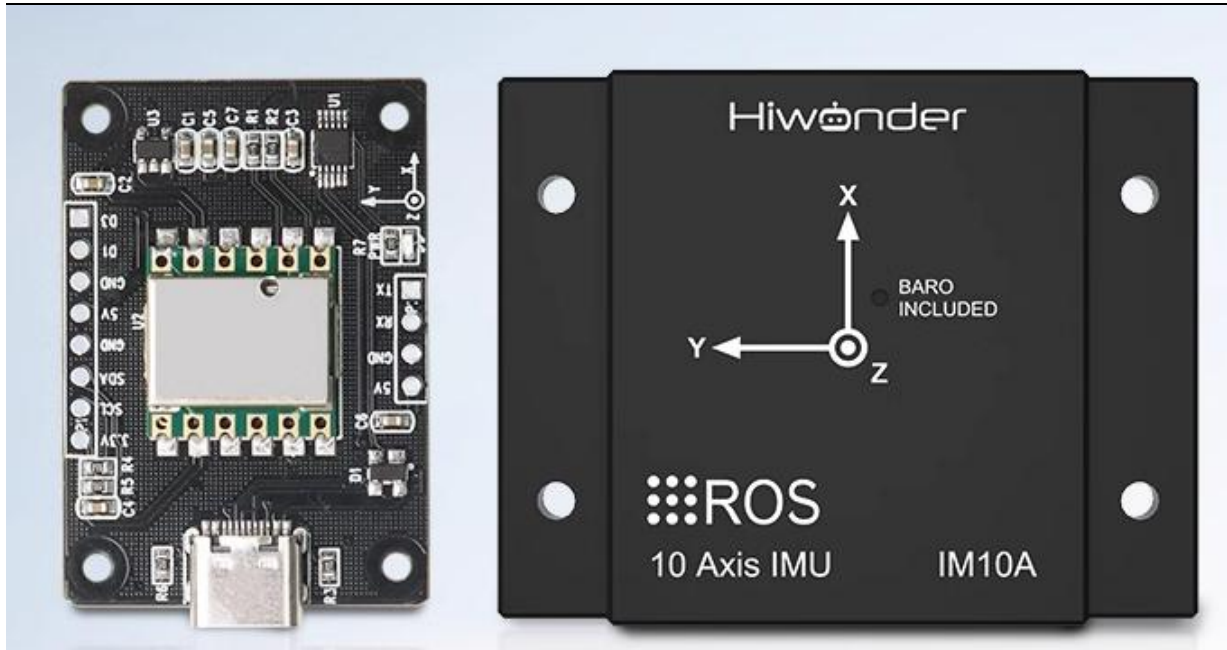


幻尔科技

IMU 应用教程-ROS 应用

V1.0



版本号	修改日期	修改摘要
V1.0	20231124	初次发布

目录

1. ROS1 应用教程	3
1.1 ROS 环境安装 系统安装及环境搭建	3
1.2 绑定端口	3
1.2.1 自动绑定端口号	3
1.2.3 手动修改 USB 设备号	4
1.3 IMU 软件包使用	5
1.3.1 编译工作空间	5
1.3.2 修改参数配置	6
1.3.3 运行可视化界面	7
2. ROS2 应用教程	8
2.1 绑定端口	9
2.1.1 自动绑定端口号	9
2.1.3 手动修改 USB 设备号	9
2.2 功能包使用	11
2.2.1 编译工作空间	11
2.2.2 修改参数配置	11
2.3 运行测试	12
3. 融合 IMU 和 GPS 数据	12

1. ROS1 应用教程

1.1 ROS 环境安装 系统安装及环境搭建

本例程默认视为系统已经安装好 ROS 开发环境，如果没有安装 ROS 环境，请自行安装好 ROS 环境再来运行此例程。

请在终端运行对应的命令

1. 如果你使用的是 ubuntu 16.04, ROS kinetic, python2 :

安装 IMU 数据处理工具和可视化工具:

```
"sudo apt-get install ros-kinetic-imu-tools ros-kinetic-rviz-imu-plugin"
```

```
"sudo apt-get install python-visual"
```

2. 如果你使用的是 ubuntu 18.04, ROS Melodic, python2 :

安装 IMU 数据处理工具和可视化工具:

```
"sudo apt-get install ros-melodic-imu-tools ros-melodic-rviz-imu-plugin"
```

3. 如果你使用的是 ubuntu 20.04, ROS Noetic, python3 :

安装 IMU 数据处理工具和可视化工具:

```
"sudo apt-get install ros-noetic-imu-tools ros-noetic-rviz-imu-plugin"
```

安装串行端口通信库:

```
"pip3 install pyserial"
```

4. 安装 ROS 串口驱动:

```
"sudo apt-get install ros-$ROS_DISTRO-serial"
```

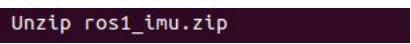
以下内容以 ubuntu 18.04, ROS Melodic 版本为例。

1.2 绑定端口

1.2.1 自动绑定端口号

先将 IMU 模块连接到电脑虚拟机的 USB 口，再将资料中的示例程序 ros1_imu.zip 复制放到用户目录。运行以下命令解压：

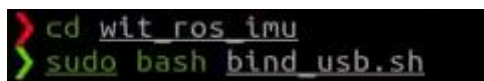
```
"unzip ros1_imu.zip"
```



```
Unzip ros1_imu.zip
```

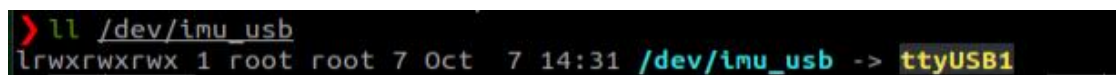
```
"cd ~/wit ros imu"
```

```
"sudo bash bind_usb.sh"
```



```
> cd wit ros imu
> sudo bash bind_usb.sh
```

重新插拔 usb 之后。后查看端口是否已经绑定完成。输入指令：“ll /dev/imu_usb”。



```
> ll /dev/imu_usb
lrwxrwxrwx 1 root root 7 Oct  7 14:31 /dev/imu_usb -> ttyUSB1
```

注意：如果上述操作未能显示绑定的端口号，插拔 usb 接头，两次输入“lsusb”指令，检查设备的厂商 ID 和产品 ID 对不对，在~/wit_ros_imu/imu_usb.rules 文件进行相应的更改。

```
> lsusb
Bus 002 Device 002: ID 0bda:0411 Realtek Semiconductor Corp.
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 8087:0a2b Intel Corp.
Bus 001 Device 009: ID 2bc5:0559
Bus 001 Device 012: ID 2bc5:0659
Bus 001 Device 006: ID 1a40:0101 Terminus Technology Inc. Hub
Bus 001 Device 005: ID 1a86:55d4 QinHeng Electronics
Bus 001 Device 011: ID 1a86:7523 QinHeng Electronics HL-340 USB-Serial adapter
Bus 001 Device 010: ID 0c76:1203 JMtek, LLC.
Bus 001 Device 013: ID 10d6:b003 Actions Semiconductor Co., Ltd
Bus 001 Device 004: ID 1a86:8091 QinHeng Electronics
Bus 001 Device 002: ID 0bda:5411 Realtek Semiconductor Corp.
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
> lsusb
Bus 002 Device 002: ID 0bda:0411 Realtek Semiconductor Corp.
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 8087:0a2b Intel Corp.
Bus 001 Device 009: ID 2bc5:0559
Bus 001 Device 012: ID 2bc5:0659
Bus 001 Device 006: ID 1a40:0101 Terminus Technology Inc. Hub
Bus 001 Device 005: ID 1a86:55d4 QinHeng Electronics
Bus 001 Device 011: ID 1a86:7523 QinHeng Electronics HL-340 USB-Serial adapter
Bus 001 Device 010: ID 0c76:1203 JMtek, LLC.
Bus 001 Device 014: ID 1a86:7523 QinHeng Electronics HL-340 USB-Serial adapter
Bus 001 Device 013: ID 10d6:b003 Actions Semiconductor Co., Ltd
Bus 001 Device 004: ID 1a86:8091 QinHeng Electronics
Bus 001 Device 002: ID 0bda:5411 Realtek Semiconductor Corp.
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub

wit_ros_imu > vim imu_usb.rules
1  KERNEL=="ttyUSB*", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523" MODE:="0666", SYMLINK+="imu_usb"
```

修改完之后重新输入“sudo bash bind_usb.sh”

```
> sudo bash bind_usb.sh
```

1.2.3 手动修改 USB 设备号

如果上述自动绑定端口操作仍未能成功，则需要手动修改模块对应的 USB 设备号，具体操作如下：

1. 查看 USB 端口号。将惯导模块通过 USB-typeC 数据线连接到电脑的 USB 口，在终端输入以下命令来检测设备端口“ls /dev/ttyUSB*”，下面是插和拔的情况，确定端口号（建议每次开机要重新确认端口号）：

```
> ls /dev/ttyUSB*
/dev/ttyUSB1
> ls /dev/ttyUSB*
/dev/ttyUSB0 /dev/ttyUSB1
```

上面的情况是电脑连接多个 USB 设备，先不插惯导模块查看一次端口，再插入惯导模块后再次查看端口，对比前后两次打印出来的端口号，第二次多的端口号就是惯导模块的 USB 设备号。

2. 修改参数配置。需要修改的参数包括 USB 端口号。

```
"cd ~/ros1Imu_ws/src/wit_ros_imu/launch"
```

```
"vim rviz_and_imu.launch"
```

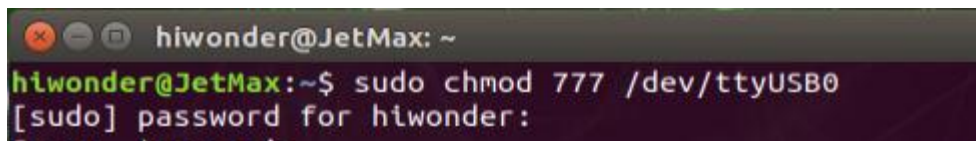
进入脚本目录 src/launch，打开 rviz_and_imu.launch 文件中的配置参数

设备号/dev/ttyUSB0（脚本默认用的 /dev/ttyUSB0），如果你的电脑识别出来的数字不是 USB0，则修改为你电脑识别到的号。注意：如果连接了多个 USB 串口设备，可能每次开机都需要手动查看并修改对应的串口号。

```
8      <!-- imu python -->
9      <node pkg="wit_ros_imu" type="wit_${arg type}_ros.py" name="imu" output="screen">
10         <param name="port" type = "str" value="/dev/ttyUSB0"/>
11         <param name="baud" type = "int" value="9600"/>
12         <remap from="/wit/imu" to="/imu/data"/>
13     </node>
```

修改完成后保存并退出。

3. 如果出现提示 USB 权限问题，请在终端输入以下命令增加权限，或者添加当前用户到 USB 设备权限列表中：**sudo chmod 777 /dev/ttyUSB0**



```
hiwonder@JetMax: ~
hiwonder@JetMax:~$ sudo chmod 777 /dev/ttyUSB0
[sudo] password for hiwonder:
```

1.3 IMU 软件包使用

1.3.1 编译工作空间

打开命令终端，运行下面指令：

1. 创建新的工作空间目录

```
"mkdir ros1Imu_ws"
```

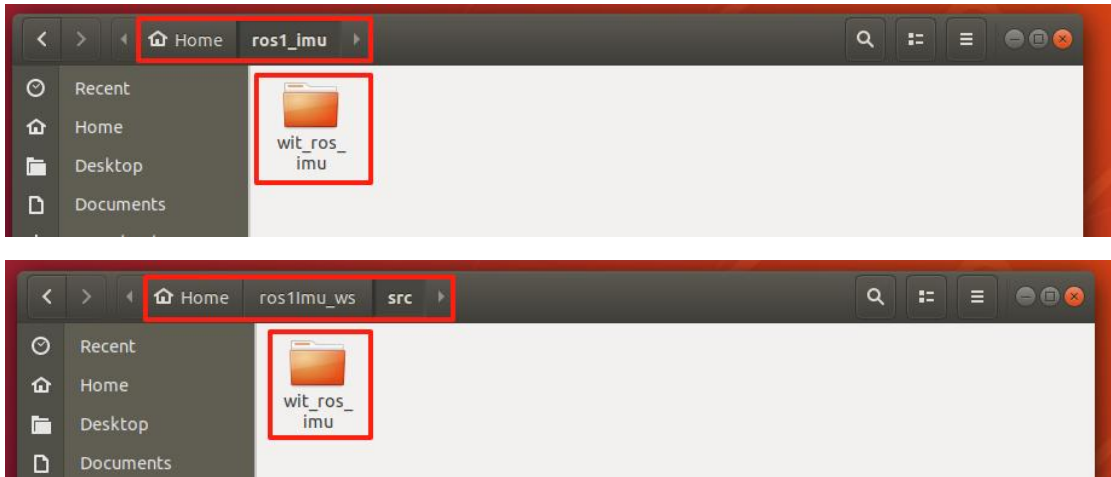
2. 进入该目录

```
"cd ros1Imu_ws"
```

3. 在工作空间内创建一个 src 目录，输入指令

```
"mkdir src"
```

4. 导入功能包，将 ros1_imu 目录下的 wit_ros_imu 复制到 src 中



5. 确保您在工作空间的根目录中

“cd ~/ros1Imu_ws”

6. 初始化工作空间

“catkin_make”

7. 添加.py 的权限

“cd ~/ros1Imu_ws/src/ros_imu_ws/src/scripts”

“sudo chmod 777 *.py”

8. 将 "source ~/ros1Imu_ws/devel/setup.sh" 添加到 ~/.zshrc 文件。

```
22 # History won't show duplicates on search.
23 source $ZSH/plugins/powerlevel10k/powerlevel10k.zsh-theme
24 source /usr/share/zsh-syntax-highlighting/zsh-syntax-highlighting.zsh
25 source $ZSH/plugins/zsh-autosuggestions/zsh-autosuggestions.zsh
26 source $HOME/ros_ws/.hiwonderrc
27 source ~/ros1Imu_ws/devel/setup.sh
```

最后输入指令**“source ~/.zshrc”**

1.3.2 修改参数配置

在路径 “~/ros1Imu_ws/src/wit_ros_imu/launch/rviz_and_imu.launch” 下，修改相对应的参数。波特率根据实际使用设定，默认波特率 baud 的值为 9600，如果用户通过上位机修改了波特率，需要对应修改成修改后的波特率。

“cd ~/ros1Imu_ws/src/wit_ros_imu/launch”

“vim rviz_and_imu.launch”

进入脚本目录 src/launch，打开 rviz_and_imu.launch 文件中的配置参数


```
8 <!-- imu python -->
9 <node pkg="wit_ros_imu" type="wit_$(arg type)_ros.py" name="imu" output="screen">
10   <param name="port" type="str" value="/dev/imu_usb"/>
11   <param name="baud" type="int" value="9600"/>
12   <remap from="/wit/imu" to="/imu/data"/>
13 </node>
```

1.3.3 运行可视化界面

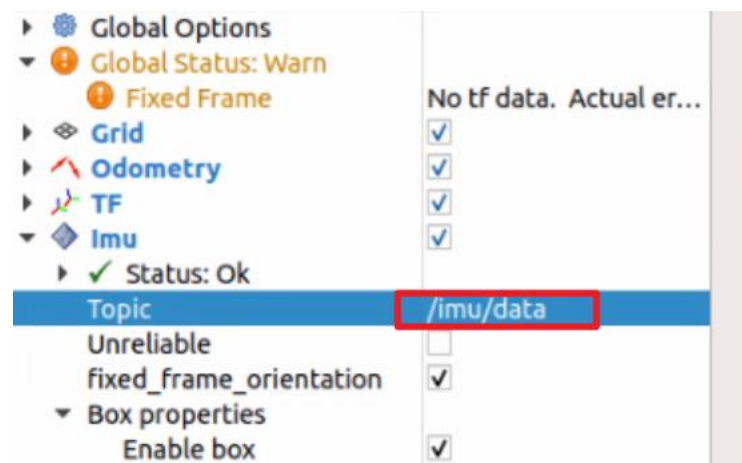
1. 首先关闭手机 APP: “`sudo systemctl stop start_app_node.service`” (在我司提供的机器人镜像里面需要关闭手机 APP 服务, 在虚拟机镜像里面没有自启服务不需要输入这条指令)

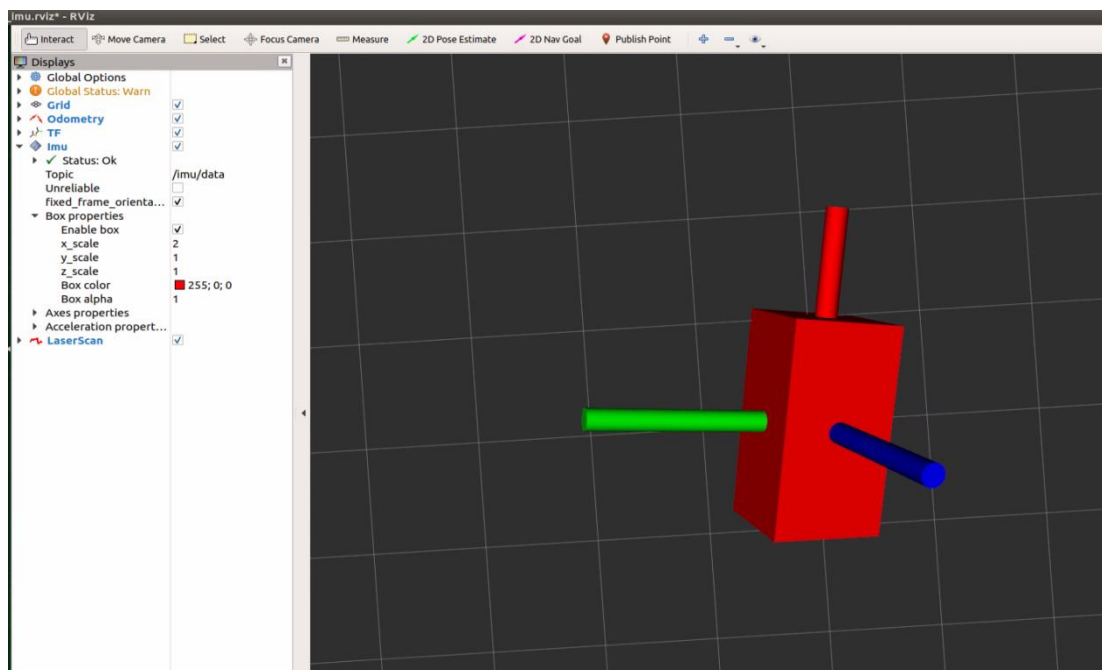
2. 打开终端, 运行 launch 文件, 注意要在实际桌面打开, 或者 VNC 远程, 不可在 SSH 远程终端中打开。

“`roslaunch wit_ros_imu rviz_and_imu.launch`”

```
roslaunch wit_ros_imu rviz_and_imu.launch
```

3. 话题选中 /imu/data, 查看模型转动效果。



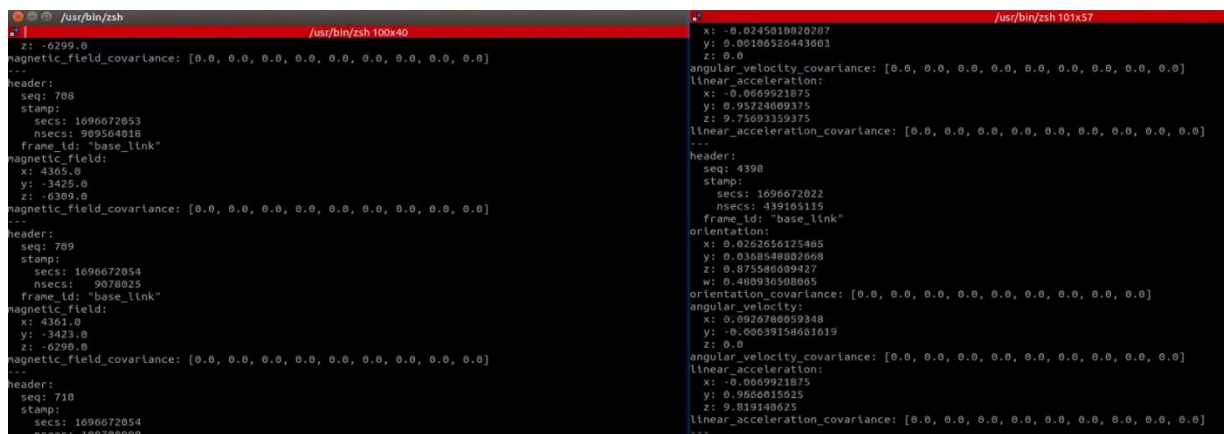


打开两个新终端输入分别输入下面两行命令，查看数据 “rostopic echo”

“rostopic echo /w/mag”

“rostopic echo /imu/data”

如下图，驱动运行成功后和输出的信息



2. ROS2 应用教程

环境搭建

本门课程说明如何在 ros2 中搭建十轴 IMU 模块环境，其中包括编译功能包、绑定串口、运行程序并查看数据。本节以 ubuntu20.04+ros-foxy，功能空间名字以 ros1_ws 为例，默认波特率是 9600。

2.1 绑定端口

2.1.1 自动绑定端口号

先将 IMU 模块连接到电脑的 USB 口,再将资料中的示例程序 `ros_imu2_ws.zip` 复制放到用户目录。运行以下命令解压:

```
“unzip wit_ros2_imu.zip”
```

```
unzip wit_ros2_imu.zip
```

```
“cd ~/wit_ros2_imu”
```

```
cd ~/wit_ros2_imu
```

```
“sudo bash bind_usb.sh”
```

```
sudo bash bind_usb.sh
```

重新插拔 usb 先之后。后查看端口是否已经绑定完成。输入指令 `ll /dev/imu_usb`

```
nano@nano-desktop:~$ ll /dev/imu_usb  
lrwxrwxrwx 1 root root 7 10月 10 15:08 /dev/imu_usb -> ttyUSB0
```

注意: 如果上述操作未能显示绑定的端口号, 插拔 usb 接头, 两次输入 `lsusb` 指令, 检查设备的厂商 ID 和产品 ID 对不对, 在

`~/ros_ws/src/wit_ros2_imu/imu_usb.rules` 文件进行相应的更改。

```
src/wit_ros2_imu$ lsusb
```

```
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub  
Bus 002 Device 006: ID 1a86:7523 QinHeng Electronics HL-340 USB-Serial adapter  
Bus 002 Device 004: ID 0e0f:0008 VMware, Inc. VMware Virtual USB Mouse  
Bus 002 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub  
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse  
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
```

```
Open [v] imu_usb.rules  
~/ros2_ws/src/wit_ros2_imu  
1 KERNEL=="ttyUSB*", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", MODE=="0666", SYMLINK+="imu_usb"  
2
```

注意: 修改完成 ID 号和端口号后, 需要使用如下指令重新加载规则文件:

```
“sudo udevadm control --reload-rules”
```

```
“sudo udevadm trigger”
```

2.1.3 手动修改 USB 设备号

如果上述自动绑定端口操作仍未能成功, 则需要手动修改模块对应的 USB 设备号, 具体操作如下:

4. 查看 USB 端口号。将惯导模块通过 USB-typeC 数据线连接到电脑的 USB 口，在终端输入以下命令来检测设备端口 “ls /dev/ttyUSB*”，下面是插和拔的情况，确定端口号（建议每次开机要重新确认端口号）：

```
hiwonder@ubuntu:~/Desktop$ ls /dev/ttyUSB*  
/dev/ttyUSB0  
  
hiwonder@ubuntu:~/ros2_ws/src/wit_ros2_imu$ ll /dev/imu_usb  
lrwxrwxrwx 1 root root 12 Oct  6 06:14 /dev/imu_usb -> /dev/ttyUSB0
```

上面的情况是电脑连接多个 USB 设备，先不插惯导模块查看一次端口，再插入惯导模块后再次查看端口，对比前后两次打印出来的端口号，第二次多的端口号就是惯导模块的 USB 设备号。

5. 修改参数配置。需要修改的参数包括 USB 端口号。

```
“cd ~/ros2_ws/src/wit_ros2_imu/launch”
```

```
“vim rviz_and_imu.launch.py”
```

进入脚本目录 src/launch，打开 rviz_and_imu.launch.py 文件中的配置参数

设备号/dev/ttyUSB0（脚本默认用的 /dev/ttyUSB0），如果你的电脑识别出来的数字不是 USB0，则修改为你电脑识别到的号。注意：如果连接了多个 USB 串口设备，可能每次开机都需要手动查看并修改对应的串口号。

```
5  def generate_launch_description():  
6      rviz_and_imu_node = Node(  
7          package='wit_ros2_imu',  
8          executable='wit_ros2_imu',  
9          name='imu',  
10         remappings=[('/wit/imu', '/imu/data')],  
11         parameters=[{'port': '/dev/ttyUSB0'},  
12                     {"baud": 9600}],  
13         output="screen"  
14     )
```

修改完成后保存并退出。

4. 如果出现提示 USB 权限问题，请在终端输入以下命令增加权限，或者添加当前用户到 USB 设备权限列表中：sudo chmod 777 /dev/ttyUSB0

```
hiwonder@JetMax: ~  
hiwonder@JetMax:~$ sudo chmod 777 /dev/ttyUSB0  
[sudo] password for hiwonder:
```

2.2 功能包使用

2.2.1 编译工作空间

建立一个工作空间 `ros_ws` 并且在该目录下新建一个 `src` 文件夹存放功能包, 终端输入,

```
Mkdir ros_ws
```

```
cd ros_ws
```

```
mkdir src
```

然后, 解压文件, 得到 `wit_ros2_imu` 文件夹, 把它复制到刚才建立的 `src` 目录下, 然后回到工作空间目录下, 使用 `colcon build` 命令进行编译, 终端输入,

```
cd ~/ros_ws
```

```
sudo colcon build
```

然后把工作空间的路径加入到 `.bashrc` 中, 终端输入,

```
sudo vim ~/.bashrc
```

`source ~/ros_ws/install/setup.bash` #把这句加在最后边, 这里的我工作空间是在~目录下的, 根据自己的工作空间目录进行修改。

2.2.2 修改参数配置

程序默认是使用 9600 的波特率, 如果在上位机修改了波特率, 那么则需要修改源码中的波特率, 源码修改波特率的位置是, `~/ros_ws/src/wit_ros2_imu/launch`

把 9600 改成上位机上修改的波特率, 然后保存后退出, 最后回到工作空间目录下进行编译即可。

在路径 “`~/ros_ws/src/wit_ros2_imu/launch/rviz_and_imu.launch`” 下的文件, 修改相对应的参数。波特率根据实际使用设定, 默认波特率 `baud` 的值为 9600, 如果用户通过上位机修改了波特率, 需要对应修改成修改后的波特率。

```
“cd ~/ros_ws/src/wit_ros2_imu/launch”
```

```
“vim rviz_and_imu.launch.py”
```

进入脚本目录 `src/launch`, 打开 `rviz_and_imu.launch.py` 文件中的配置参数

注意:

1. 修改程序之后要重新编译否则可能不生效: `sudo colcon build`

2. 更改目录权限: `sudo chmod -R 755 /path/ros_ws/src`

2.3 运行测试

测试之前需要安装库：sudo pip install pyserial

终端输入，ros2 launch wit_ros2_imu rviz_and_imu.launch.py

```
hiwonder@ubuntu:~/ros_ws/src$ ros2 launch wit_ros2_imu rviz_and_imu.launch.py
[INFO] [launch]: All log files can be found below /home/hiwonder/.ros/log/2023-10-06-03-29-30-618005-ubuntu-3318
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [wit_ros2_imu-1]: process started with pid [3380]
[wit_ros2_imu-1] [INFO] [1696588174.438522511] [imu]: Serial port opened successfully...
```

使用 ros2 topic echo 工具可以看发布的数据的具体内容，终端输入，

“ros2 topic echo /imu/data_raw”

```
ros2 topic echo /imu/data_raw
```

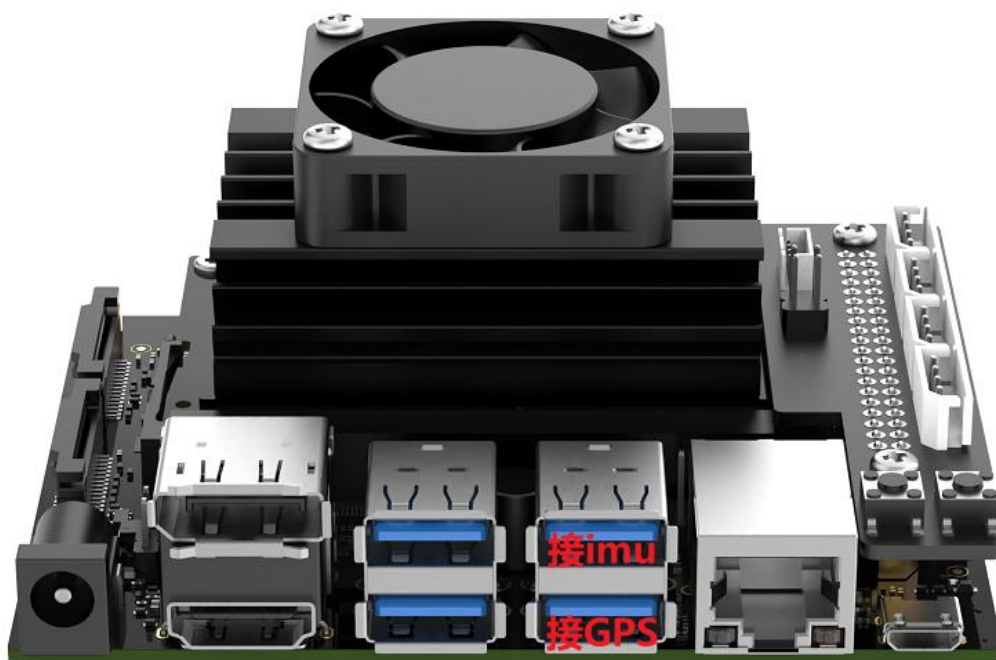
显示内容下图所示：

```
header:
  stamp:
    sec: 1708425941
    nanosec: 88615768
  frame_id: imu_link
orientation:
  x: 0.13942420691484847
  y: -0.9900991489617085
  z: -0.002775917342391727
  w: 0.016026853539431774
orientation_covariance:
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
angular_velocity:
  x: 0.0
  y: 0.0
  z: 0.0
angular_velocity_covariance:
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
linear_acceleration:
  x: 0.0002546787261962891
  y: -9.34600830078125e-06
  z: -0.0047688007354736335
linear_acceleration_covariance:
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
  0.0
---
```

3. 融合 IMU 和 GPS 数据

3.1 模块连接

这个的端口区分是采用 jetson nano 主控举例，使用电脑虚拟机或树莓派主控，可查询修改对应的备标识号。



3.2 端口区分

1) 导入 gps_ws 和 imu_ws 软件包到 jetson nano 的桌面。

2) 终端输入 `lsusb` 查找 GPS 和 IMU 模块连接的设备 ID 号，如下图所示，我们可以看到两者的 ID 号是一样的。

```
> lsusb
Bus 002 Device 002: ID 0bda:0411 Realtek Semiconductor Corp.
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 8087:0a2b Intel Corp.
Bus 001 Device 007: ID 1a86:7523 QinHeng Electronics HL-340 USB-Serial adapter
Bus 001 Device 005: ID 1a86:7523 QinHeng Electronics HL-340 USB-Serial adapter
Bus 001 Device 004: ID 1a86:55d4 QinHeng Electronics
Bus 001 Device 002: ID 0bda:5411 Realtek Semiconductor Corp.
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

3) 记下来进行进一步的区分,使用设备在 USB 总线上的地址进行区分,先是只插入 imu 模块,输入指令 `udevadm info -a -n /dev/ttyUSB*` , 如下图显示我们可以看到 imu 的在此机器人的唯一设备标识号为“1-2.3:1.0”。

```
> udevadm info -a -n /dev/ttyUSB*
```



```

looking at device '/devices/70090000.xusb/usb1/1-2/1-2.3/1-2.3:1.0/ttyUSB0/tty
/ttyUSB0':
  KERNEL=="ttyUSB0"
  SUBSYSTEM=="tty"
  DRIVER==" "

looking at parent device '/devices/70090000.xusb/usb1/1-2/1-2.3/1-2.3:1.0/ttyU
SB0':
  KERNELS=="ttyUSB0"
  SUBSYSTEMS=="usb-serial"
  DRIVERS=="ch341-uart"
  ATTRS{port_number}=="0"

looking at parent device '/devices/70090000.xusb/usb1/1-2/1-2.3/1-2.3:1.0':
  KERNELS=="1-2.3:1.0"
  SUBSYSTEMS=="usb"
  DRIVERS=="ch341"

```

4) 接着拔除 imu 模块的 USB 线，插入 gps 模块的 USB，输入指令：

`udevadm info -a -n /dev/ttyUSB*` 可以看到此时设备的唯一标识为 “1-2.4:1.0”

```
> udevadm info -a -n /dev/ttyUSB*
```

```

looking at device '/devices/70090000.xusb/usb1/1-2/1-2.4/1-2.4:1.0/ttyUSB0/tty
/ttyUSB0':
  KERNEL=="ttyUSB0"
  SUBSYSTEM=="tty"
  DRIVER==" "

looking at parent device '/devices/70090000.xusb/usb1/1-2/1-2.4/1-2.4:1.0/ttyU
SB0':
  KERNELS=="ttyUSB0"
  SUBSYSTEMS=="usb-serial"
  DRIVERS=="ch341-uart"
  ATTRS{port_number}=="0"

looking at parent device '/devices/70090000.xusb/usb1/1-2/1-2.4/1-2.4:1.0':
  KERNELS=="1-2.4:1.0"
  SUBSYSTEMS=="usb"
  DRIVERS=="ch341"

```

5) 在 imu_usb.rules 文件添加设备标识左区分，输入指令 `sudo vim /etc/udev/rules.d/imu_usb.rules` 在文档里输入下面的内容：

```

SUBSYSTEM=="tty", KERNELS=="1-2.3:1.0", MODE:="0666",
SYMLINK+="imu_usb"

```

```

/usr/bin/zsh 101x28
1 SUBSYSTEM=="tty", KERNELS=="1-2.3:1.0", MODE:="0666", SYMLINK+="imu_usb

```

6) 在 gps.rules 文件添加设备标识左区分，输入指令 `sudo vim /etc/udev/rules.d/gps.rules` 在文档里输入下面的内容：


```
SUBSYSTEM=="tty", KERNELS=="1-2.4:1.0", MODE:="0666",  
SYMLINK+="gps_serial"
```

```
1 SUBSYSTEM=="tty", KERNELS=="1-2.4:1.0", MODE:="0666",SYMLINK+="gps_serial
```

7) 修改完之后, 输入下面两句指令, 重新加载 udev 规则, 并触发设备重新识别:

```
sudo udevadm control --reload-rules
```

```
sudo udevadm trigger
```

```
sudo udevadm control --reload-rules  
sudo udevadm trigger
```

imu 和 gps 模块都插上验证:

```
> ll /dev/imu_usb  
lrwxrwxrwx 1 root root 7 Nov 27 19:39 /dev/imu_usb -> ttyUSB1  
> ll /dev/gps_serial  
lrwxrwxrwx 1 root root 7 Nov 27 19:37 /dev/gps_serial -> ttyUSB0
```

拔掉 imu 模块验证:

```
> ll /dev/imu_usb  
ls: cannot access '/dev/imu_usb': No such file or directory  
> ll /dev/gps_serial  
lrwxrwxrwx 1 root root 7 Nov 27 19:37 /dev/gps_serial -> ttyUSB0
```

验证无误后进行下面的内容。

3.3 编译工作空间

1) 将 gps_ws 和 imu_ws 功能包放在主目录下, 分别进入到 gps_ws 和 imu_ws, 输入指令进行编译:

```
catkin_make
```

```
~/gps_ws  
catkin_make
```

```
~/imu_ws  
catkin_make
```

2) 将下面内容添加到 ~/.zshrc 文件。

```
source ~/imu_ws/devel/setup.sh
source ~/gps_ws/devel/setup.sh
25 source $ZSH/plugins/zsh-autosuggestions/zsh-autosuggestions.zsh
26 source $HOME/ros_ws/.hiwonderrc
27 source ~/imu_ws/devel/setup.sh
28 source ~/gps_ws/devel/setup.sh
29
```

3) 最后输入指令刷新环境变量：

```
source ~/.zshrc
```

3.4 融合 IMU 和 GPS 数据

该功能融合 IMU 和 GPS 数据，并且在 rviz 中展示融合后的数据。 注意以下几点：

1) 该功能需要将购买的 IMU 和 GPS 模块通过两条数据线接在主控上。

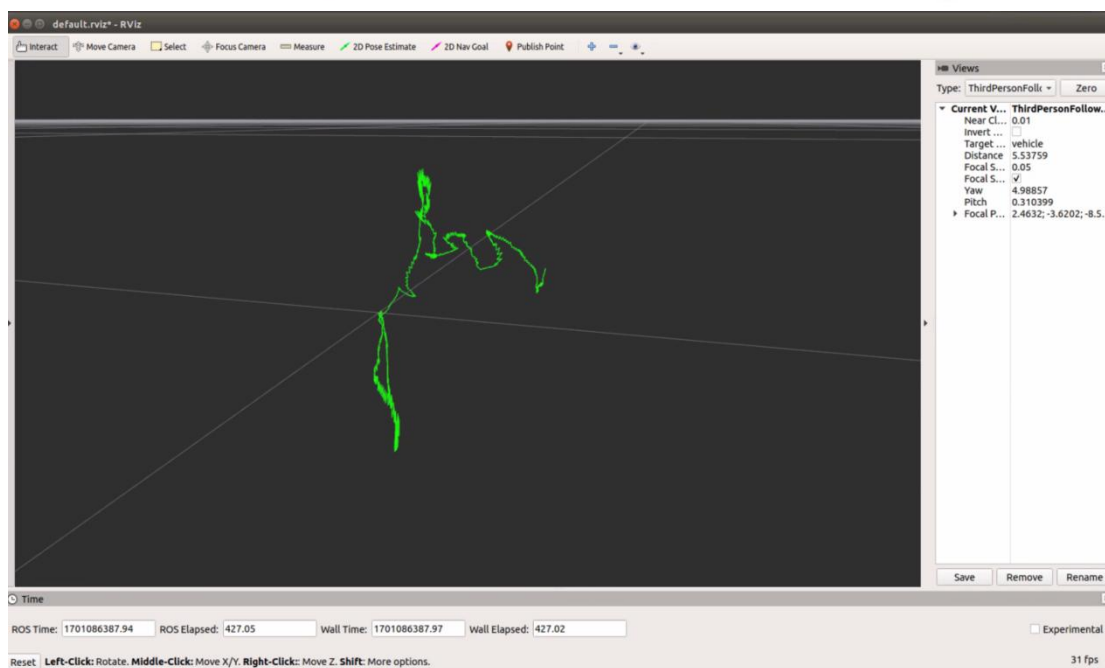
2) 需要有两个工作空间，gps_ws 和 imu_ws，它们的源码在当前目录下，对应的是 gps_src 和 imu_src。gps_src 解压后，把 gps_src 里边的 src 里边的内容复制到 gps_ws 的 src 中，然后使用 catkin_make 编译它；imu_src 解压后，把 imu_src 里边的 src 里边的内容复制到 imu_ws 的 src 中，然后使用 catkin_make 编译它。这一步骤可以参看虚拟机里边的 gps_ws 和 imu_ws。

3) Imu 和 gps 模块的波特率都是 9600

数据融合需要有两个话题的数据，/imu/data 和 /fix。这两个数据我们可以通过启动 IMU 和 GPS 模块来获取。终端输入指令：

```
roslaunch imu_gps_localization imu_gps_test.launch
```

4) 程序启动后，会提示出不够 IMU 数据，这时候只需要耐心等待一下，过一会儿就会看到这个警告 消失了，随后就会打印出数据融合后的路径。下图为实现的效果图片：



5) launch 的具体内容如下图所示：

```
<launch>
  <param name="acc_noise" type="double" value="1e-2" />
  <param name="gyro_noise" type="double" value="1e-4" />
  <param name="acc_bias_noise" type="double" value="1e-6" />
  <param name="gyro_bias_noise" type="double" value="1e-8" />

  <param name="I_p_Gps_x" type="double" value="0.0" />
  <param name="I_p_Gps_y" type="double" value="0.0" />
  <param name="I_p_Gps_z" type="double" value="0.0" />
  <param name="log_folder" type="string" value="$(find imu_gps_localization)" />

  <!--GPS-->
  <include file="$(find nmea_navsat_driver)/launch/nmea_serial_driver.launch"/>

  <!--IMU-->
  <include file="$(find wit_ros_imu)/launch/rviz_and_imu.launch"/>

  <node name="imu_gps_localization_node" pkg="imu_gps_localization" type="imu_gps_localization_node" output="screen" />

  <node pkg="rviz" type="rviz" name="rviz" output="screen"
    | args="-d $(find imu_gps_localization)/ros_wrapper/rviz/default.rviz" required="true">
  </node>
</launch>
```

可以看出，启动了 `nmea_serial_driver.launch`，这是用来获取 GPS 数据并且发布 `/fix` 数据；启动了 `rviz_and_imu.launch`，这是用来获取 IMU 数据，并且发布 `/imu/data`；最后，启动了 `imu_gps_localization_node`，这个节点订阅了 `/fix` 和 `/imu/data` 话题，得到数据后，就开始进行融合，最后发布出 `/fused_path` 的数据，可以通过 `rostopic list` 查看话题列表，也可以通过 `roslaunch rqt_graph rqt_graph` 来查看节点之间的关系。话题图片如下图所示：

